



UNINASSAU

Conceitos Básicos e Terminologias de Programação Orientada a Objetos

Análise e Desenvolvimento de Sistemas
Programação Orientada à Objetos e Estruturas de dados
Profº Eng. Esp. Lindenberg Andrade

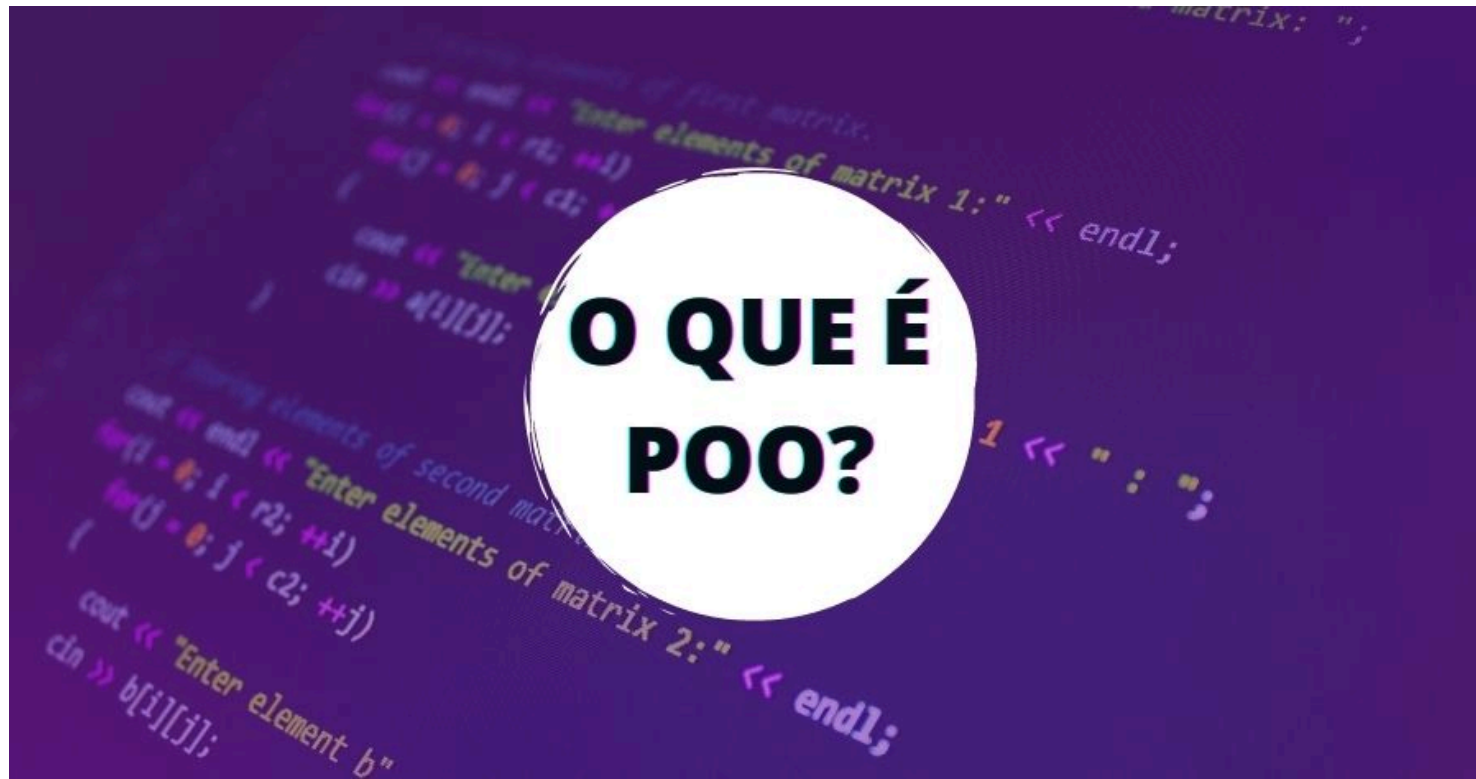
OBJETIVOS DA AULA

- Compreender a importância da Programação Orientada a Objetos (POO).
- Conhecer seus conceitos fundamentais.
- Entender diferenças em relação à programação estruturada.
- Aprender a visualizar objetos no mundo real e no código.



Introdução à POO

O QUE É PROGRAMAÇÃO ORIENTADA A OBJETOS (POO)?



- POO é um paradigma de programação que estrutura o código em torno de objetos, que representam entidades do mundo real ou conceitos abstratos, com características (atributos) e comportamentos (métodos). É uma forma de organizar o código para simplificar programas complexos, dividindo-os em unidades menores e reutilizáveis.
- Paradigma de programação baseado em **objetos**.
- Objetos possuem **atributos (dados)** e **métodos (ações)**.
- Facilita a modelagem de problemas reais.
- Utilizada em diversas linguagens modernas (Java, Python, C++, C#, etc.).

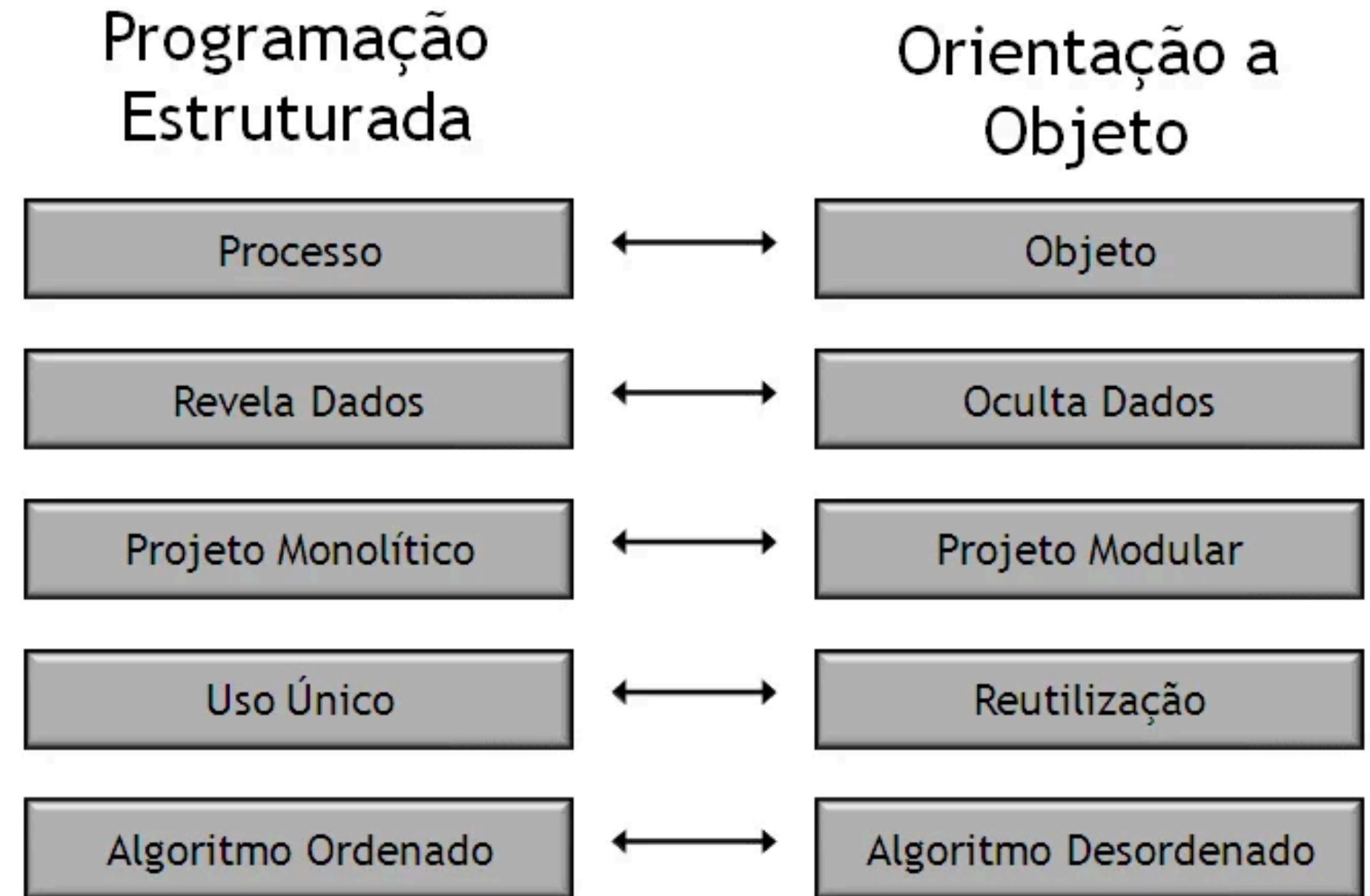
DIFERENÇA ENTRE POO E PROGRAMAÇÃO ESTRUTURADA

Programação Estruturada

- Código sequencial;
- Foco em funções e procedimentos;
- Dados e funções separados.

POO

- Código baseado em objetos;
- Foco em classes, atributos e métodos;
- Dados e comportamentos agrupados.



VANTAGENS DA POO

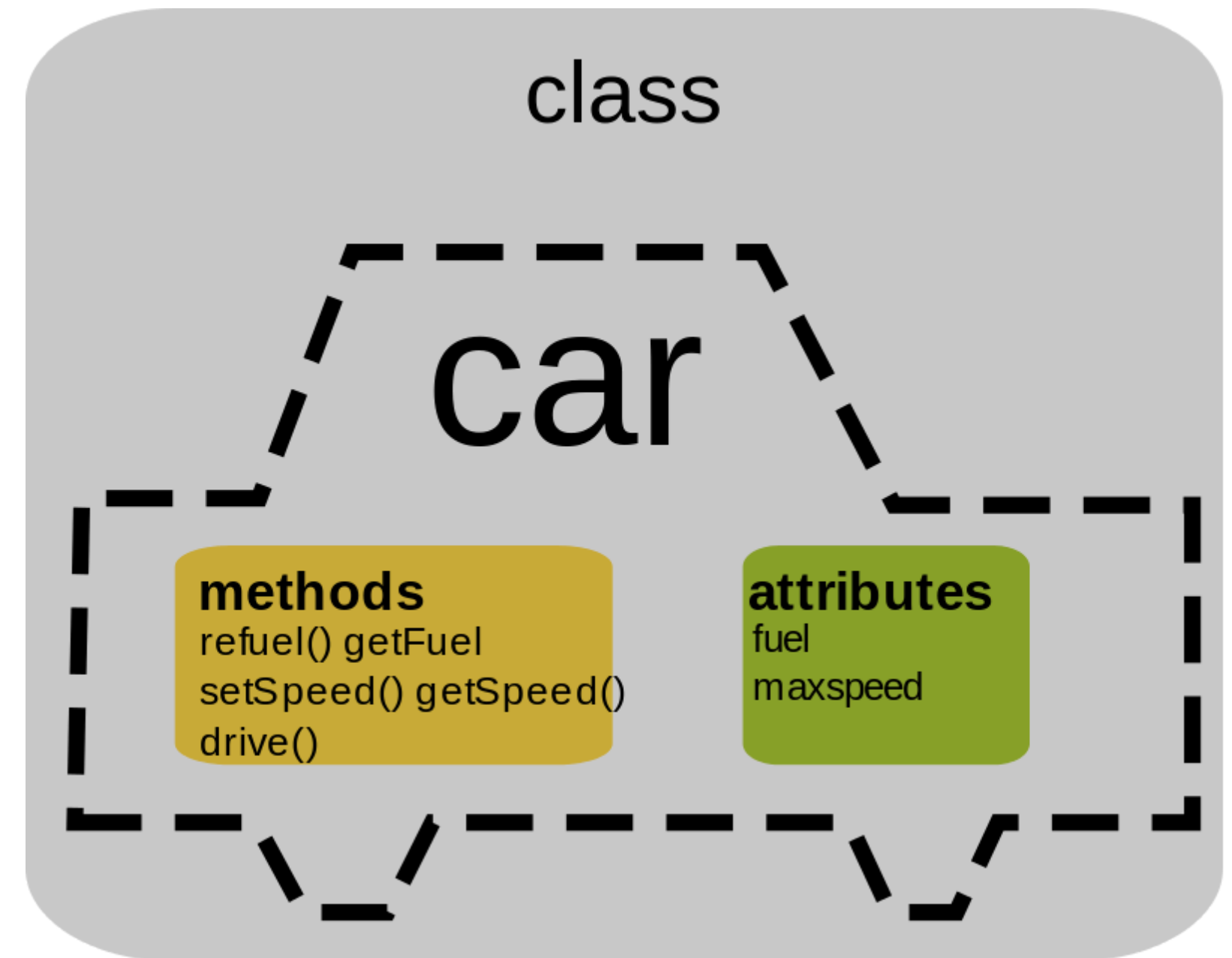
- **Reutilização** de código (herança, polimorfismo).
- **Modularidade** (organização em classes).
- **Facilidade de manutenção.**
- **Escalabilidade** em projetos grandes.
- **Abstração:** simplificação de problemas complexos.



4 pilares da POO

REUSO DE CÓDIGO E MODULARIDADE

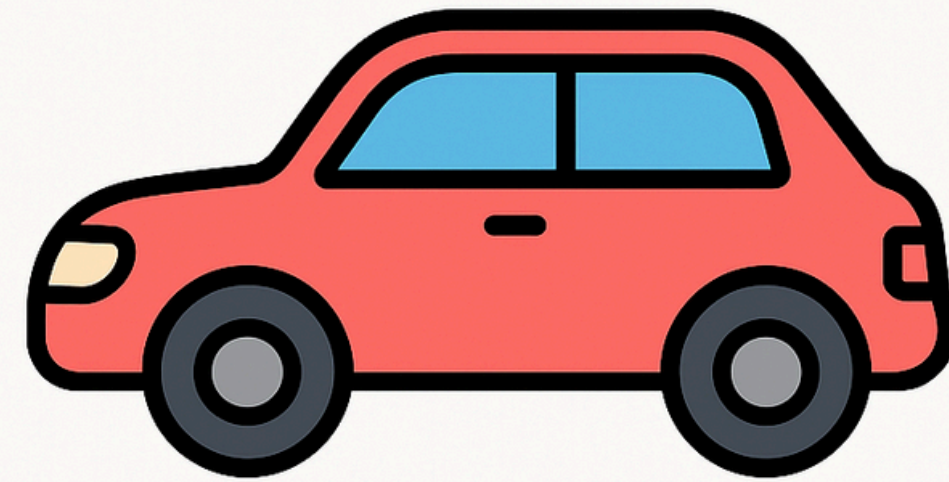
- Criar uma classe uma vez e reutilizá-la em vários programas.
- Dividir um sistema em **módulos menores (classes)**.
- Reduz duplicação de código.
- Exemplo: Classe **Carro** pode ser usada em um jogo, em um sistema de oficina, etc.



PARADIGMA DE OBJETOS NO MUNDO REAL

- POO se inspira no mundo real.
- Exemplo:
 1. Objeto: Carro
 2. Atributos: cor, modelo, ano.
 3. Métodos: ligar(), acelerar(), frear().
- Assim como na vida real, cada objeto tem estado e comportamento.

Objeto: Carro



Atributos

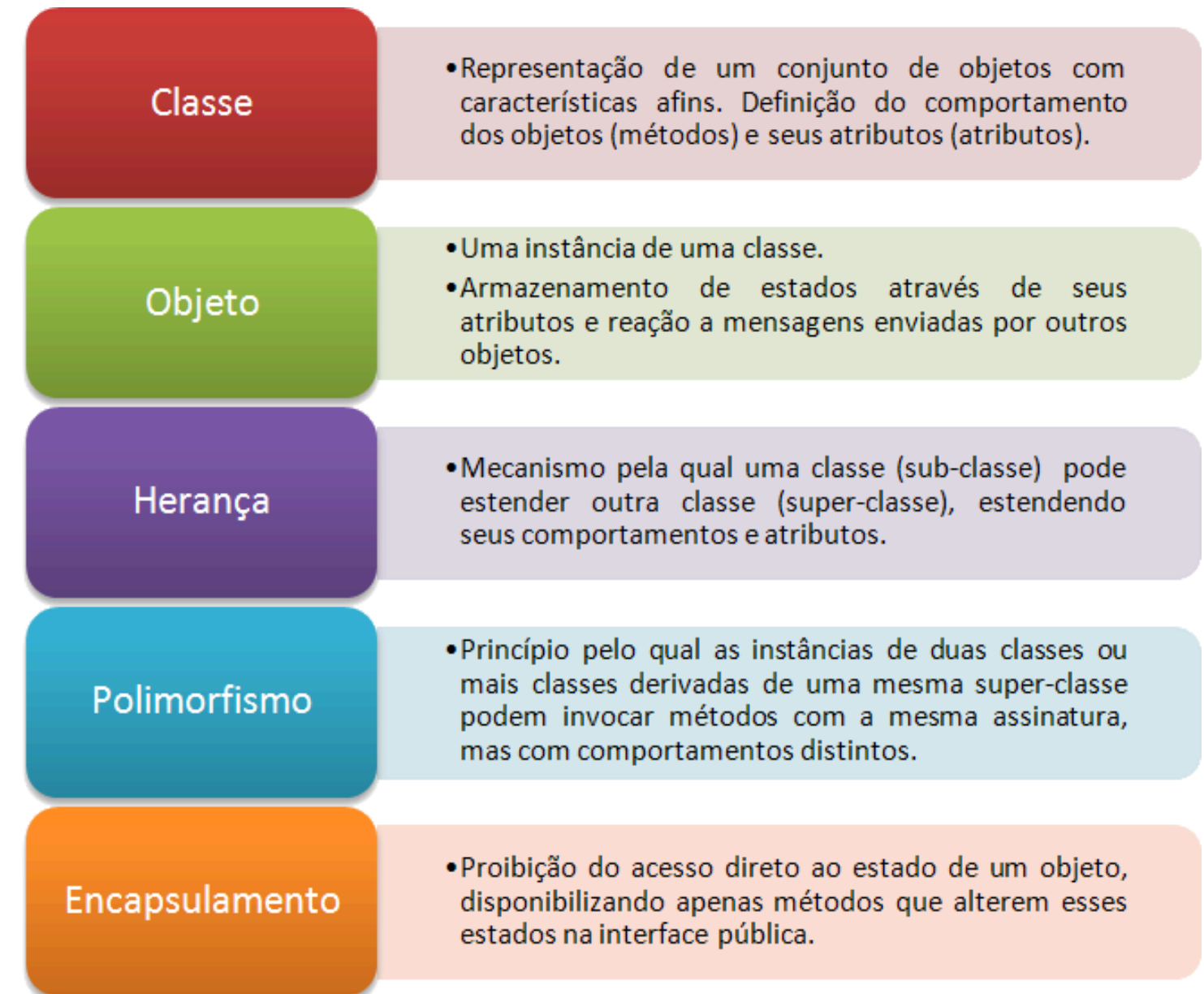
- cor
- modelo
- ano

Métodos

- ligar()
- acelerar()
- frear()

PRINCIPAIS CONCEITOS DA POO

- **Classe:** modelo/estrutura.
- **Objeto:** instância da classe.
- **Atributos:** características.
- **Métodos:** comportamentos.
- **Construtores:** inicialização de objetos.
- **Sobrecarga:** diferentes versões de métodos/construtores.



EXEMPLO PRÁTICO: CARRO



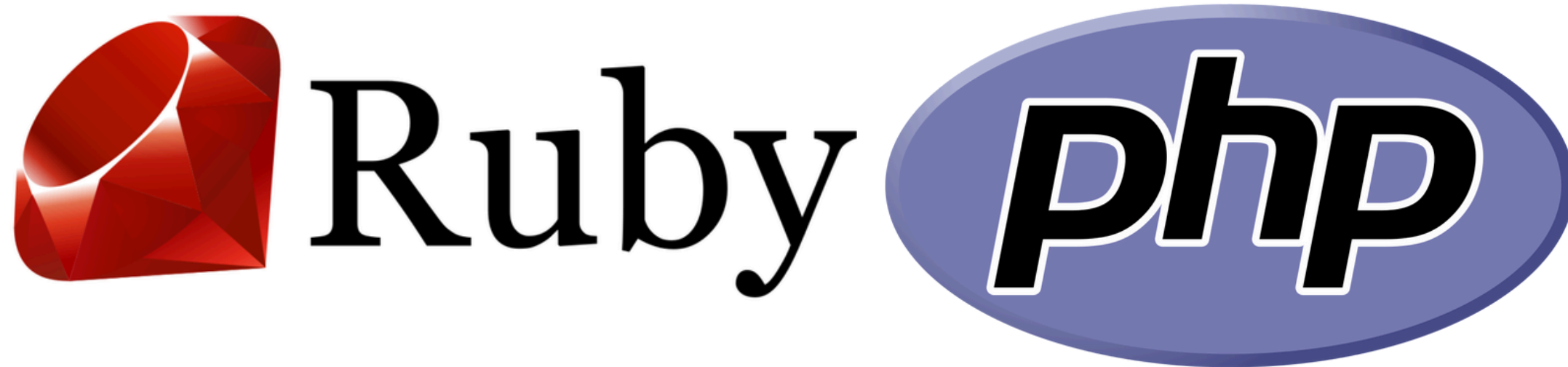
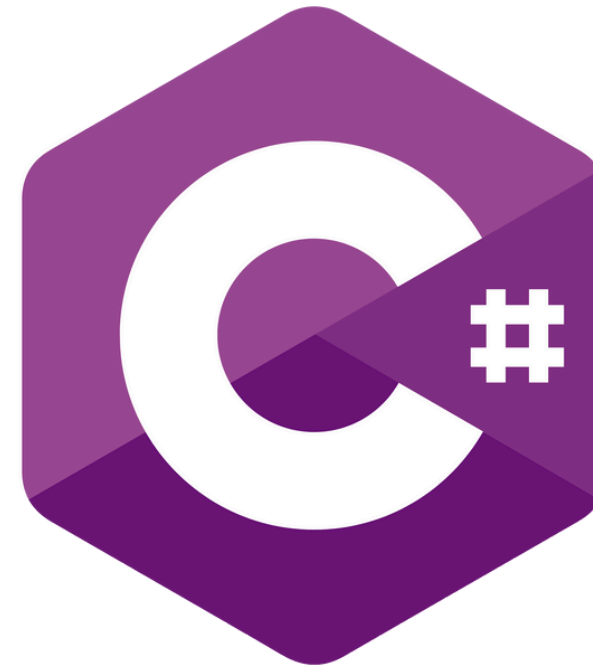
Classe: Carro

Atributos: cor, modelo, ano

Método:
ligar()

```
class Carro {  
    String cor;  
    String modelo;  
    int ano;  
  
    void ligar() {  
        System.out.println("Carro ligado!");  
    }  
}
```

LINGUAGENS QUE UTILIZAM POO



LINGUAGENS QUE UTILIZAM POO

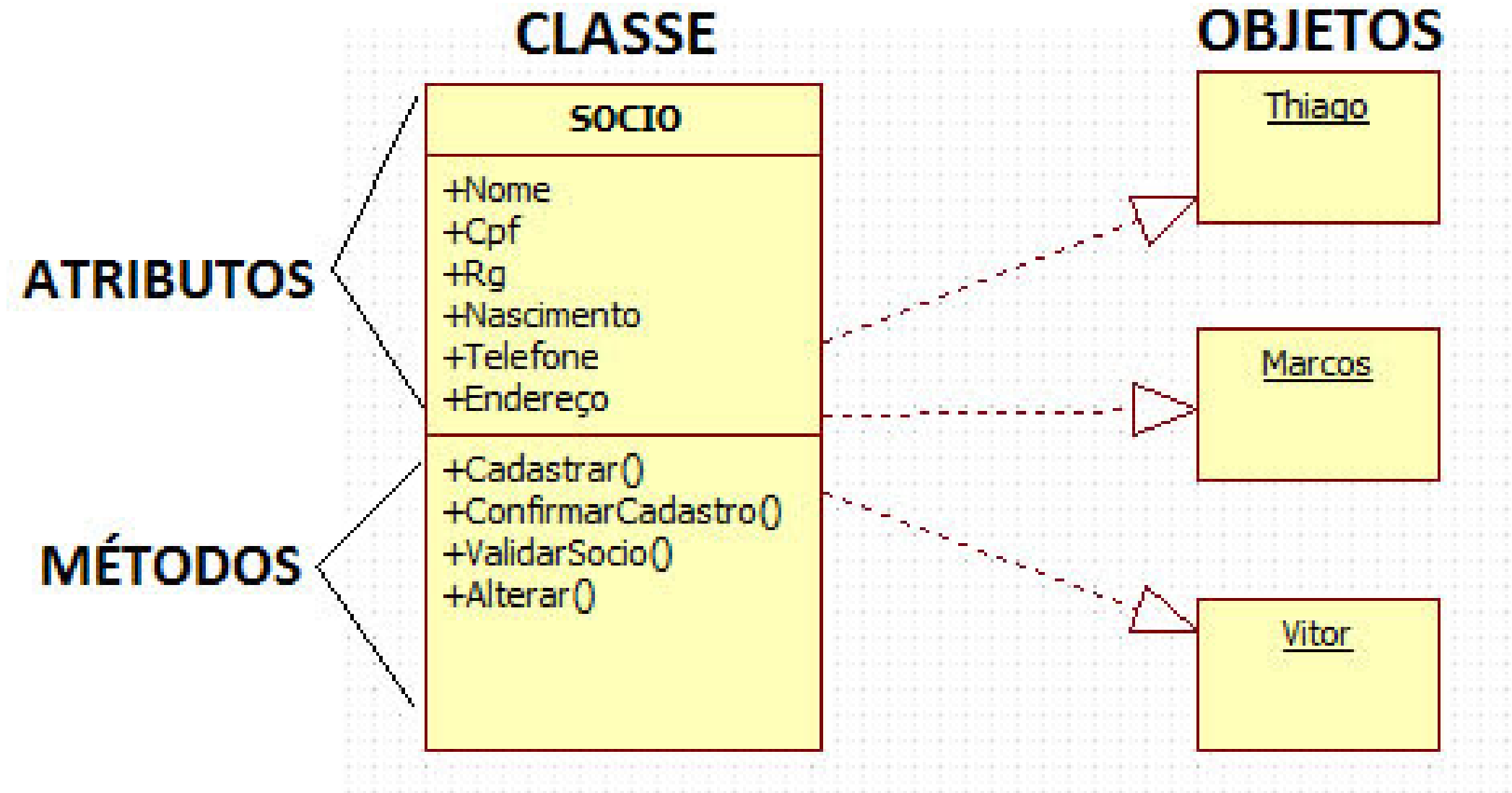
- Java → totalmente orientada a objetos.
- Python → suporta múltiplos paradigmas (inclusive POO).
- C++ → evoluiu da programação estruturada para POO.
- C# → foco em aplicações empresariais.
- Outras: Ruby, PHP, Kotlin, Swift.



Classe

O QUE É UMA CLASSE?

- Uma classe é um modelo ou molde para criar objetos;
- Define atributos (dados) e métodos (ações);
- É como uma “fábrica” que produz objetos.



ESTRUTURA BÁSICA DE UMA CLASSE

- **Classe:** Aluno
- **Atributos:** nome, idade, curso
- **Métodos:** estudar(), assistirAula()

```
class Aluno {  
    String nome;  
    int idade;  
    String curso;  
  
    void estudar() {  
        System.out.println(nome + " está estudando.");  
    }  
  
    void assistirAula() {  
        System.out.println(nome + " está assistindo à aula.");  
    }  
}
```

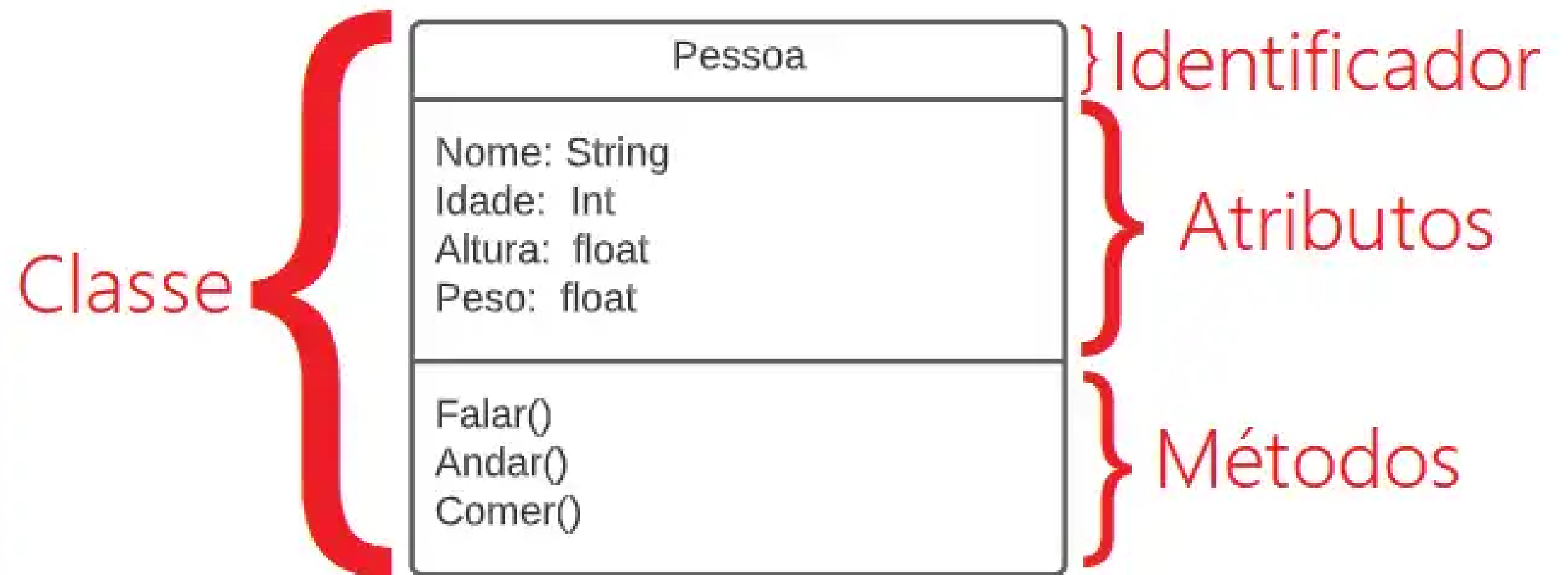
CLASSE COMO ABSTRAÇÃO

- Representa conceitos do mundo real.
- Não existe fisicamente, mas **descreve** algo.
- Exemplo: **Classe Aluno** descreve as características e comportamentos de qualquer aluno.

```
class Aluno {  
    String nome;  
    int idade;  
    String curso;  
  
    void estudar() {  
        System.out.println(nome + " está estudando.");  
    }  
  
    void assistirAula() {  
        System.out.println(nome + " está assistindo à aula.");  
    }  
}
```

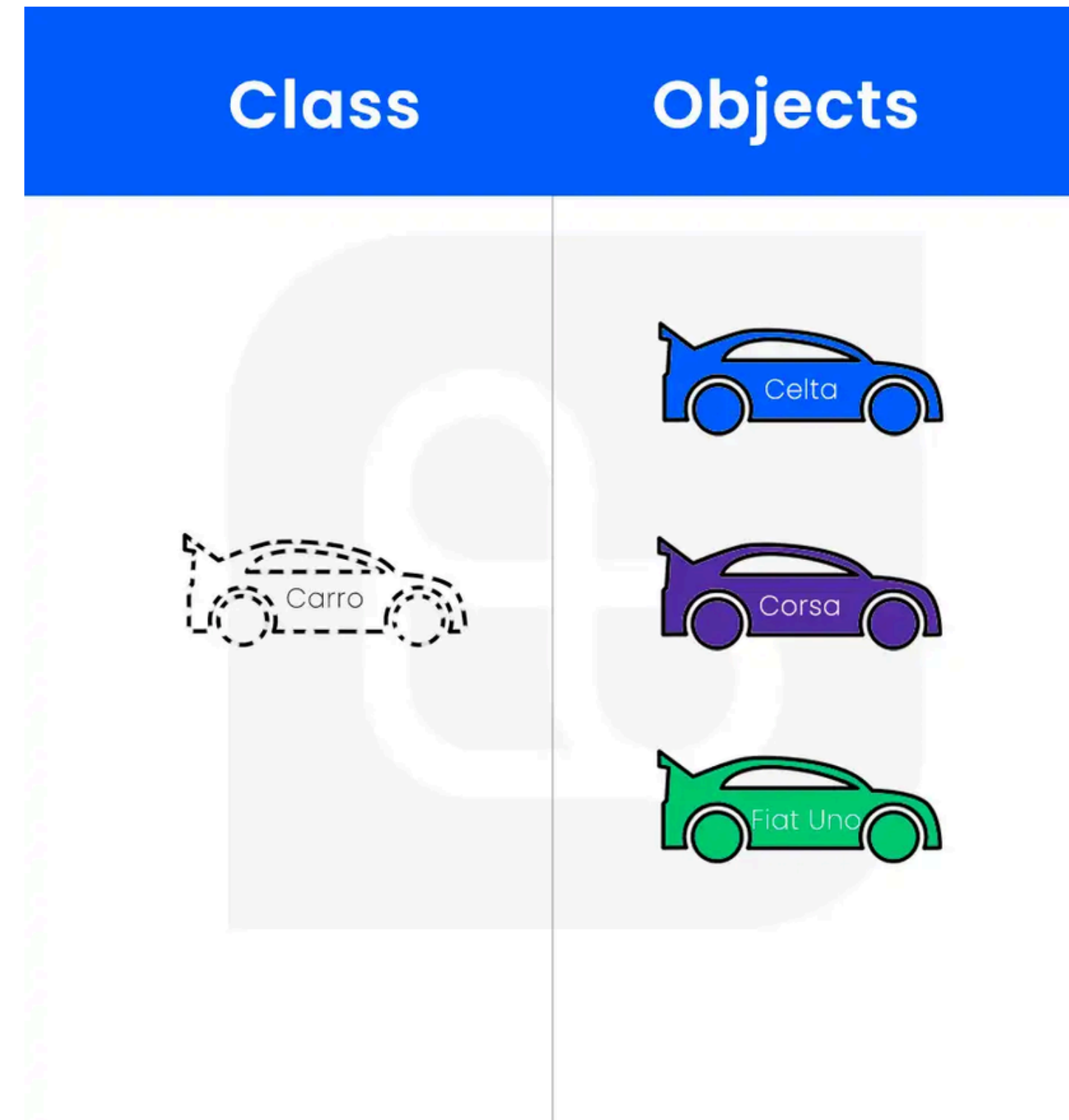

PAPEL DA CLASSE NA POO

- **Organizar** código.
- **Agrupar** dados e comportamentos.
- **Facilitar** a manutenção.
- **Reutilizar** em diferentes sistemas.



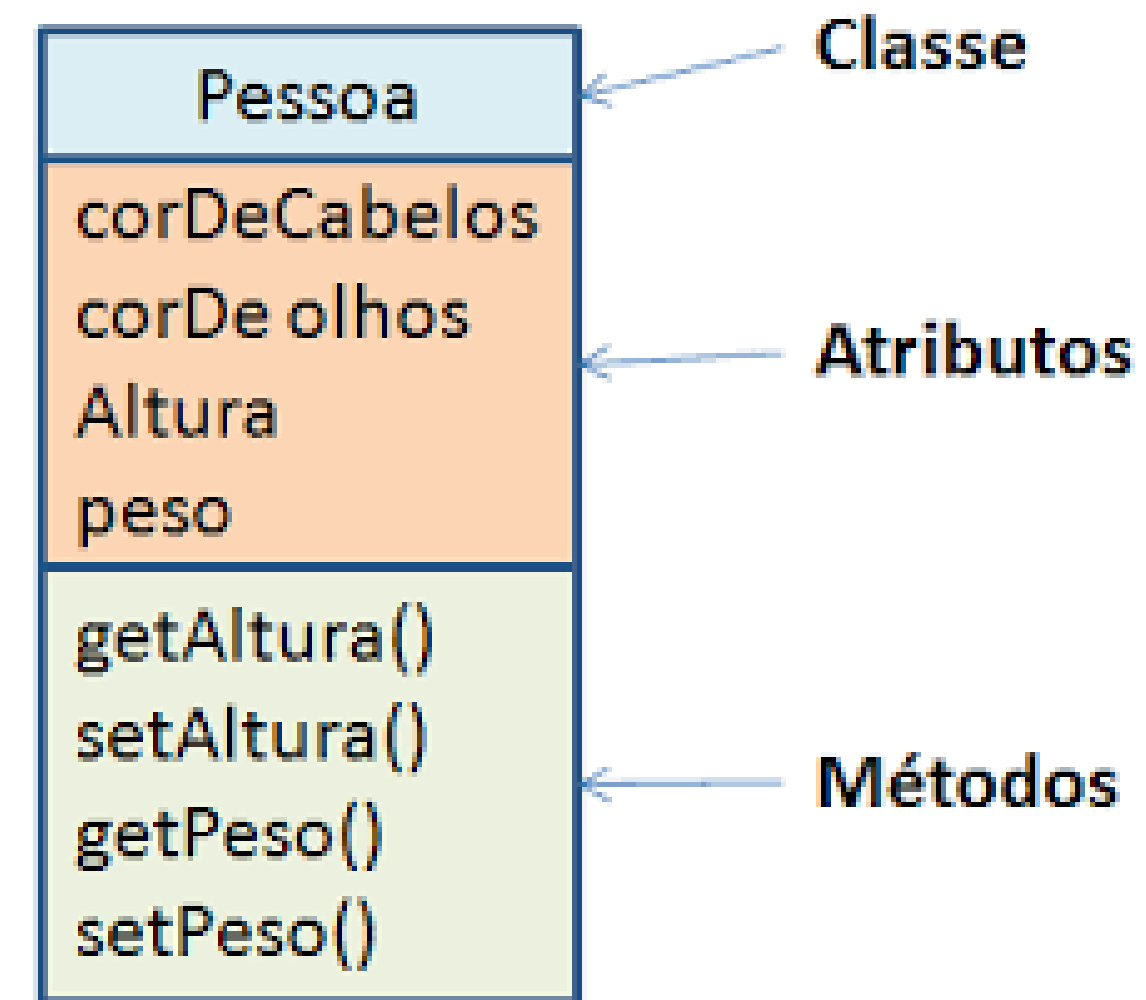
DIFERENÇA ENTRE CLASSE E OBJETO

- **Classe:** modelo, estrutura, definição.
- **Objeto:** instância da classe (criado a partir dela).
- **Exemplo:**
 - Classe: Carro
 - Objetos: carro1, carro2, carro3



BOAS PRÁTICAS EM CLASSES

- Usar **nomes significativos** (ex.: Carro, Cliente, Produto).
- Começar com letra maiúscula.
- Evitar nomes genéricos como “Teste” ou “Coisa”.
- Uma classe deve ter **responsabilidade clara**.



CLASSES NO MUNDO REAL

Exemplo:

- Classe: Aluno

1. Atributos: nome, matrícula, curso.

2. Métodos: estudar(), assistirAula().

Outro exemplo:

- Classe: ContaBancaria

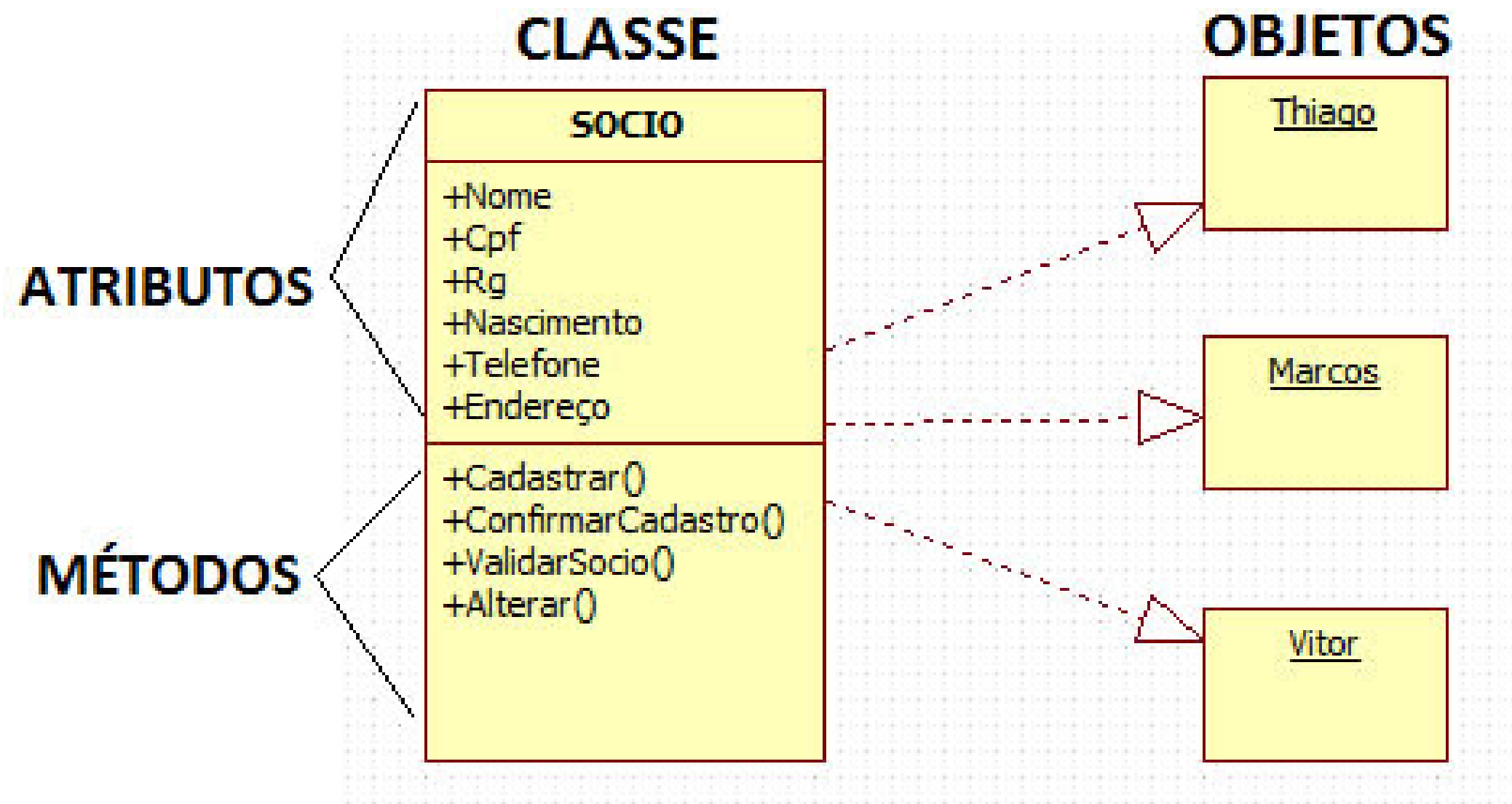
1. Atributos: saldo, número, titular.

2. Métodos: depositar(), sacar().

Objetos

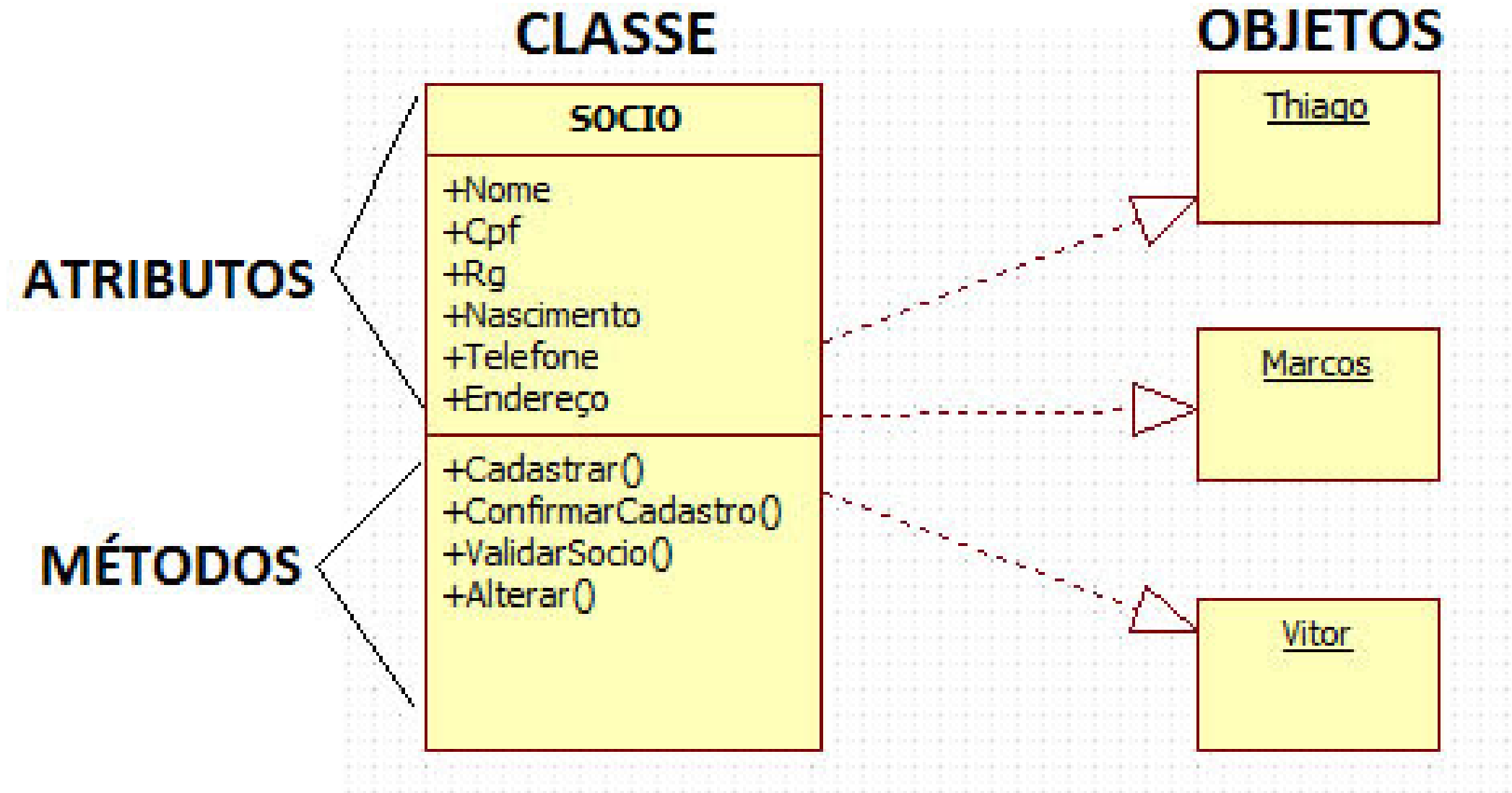
O QUE É UM OBJETO?

- Um **objeto** é uma **instância de uma classe**.
- Representa algo **concreto** no programa.
- Possui:
- **Atributos** → descrevem características.
- **Métodos** → definem comportamentos.



RELAÇÃO ENTRE CLASSE E OBJETO

- **Classe** = modelo (molde).
- **Objeto** = produto final criado a partir do molde.
- Analogia:
- Classe: planta de uma casa.
- Objeto: uma casa construída seguindo a planta.



ESTADO E COMPORTAMENTO DO OBJETO

- **Estado:** valores atuais dos atributos.
- Exemplo: nome = "Maria", curso = "Engenharia".
- **Comportamento:** ações realizadas pelos métodos.
- Exemplo: estudar(), assistirAula().



ESTADO E COMPORTAMENTO DO OBJETO

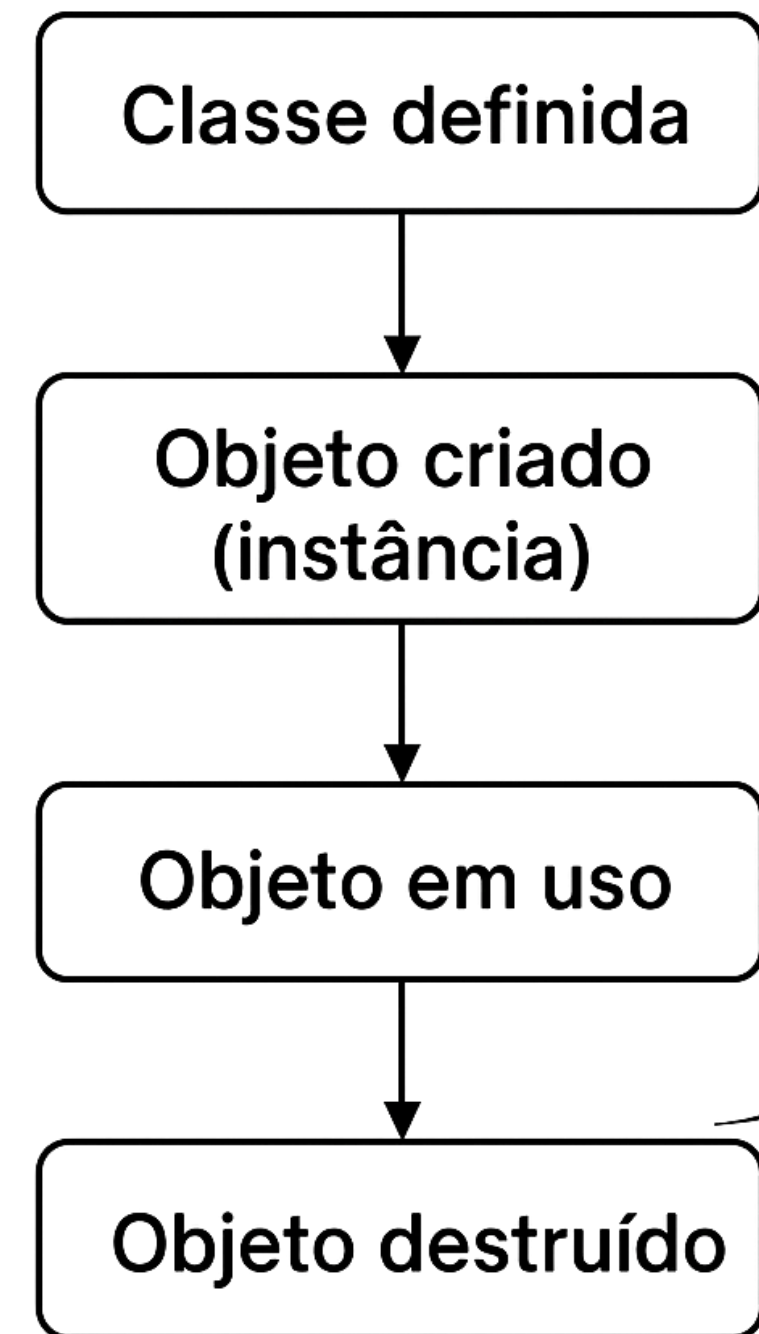
- **Estado:** valores atuais dos atributos.
- Exemplo: nome = "Maria", curso = "Engenharia".
- **Comportamento:** ações realizadas pelos métodos.
- Exemplo: estudar(), assistirAula().

Criação de Objetos em Código (Java)

```
Aluno aluno1 = new Aluno();  
aluno1.nome = "Maria";  
aluno1.idade = 20;  
aluno1.curso = "Engenharia";
```

CICLO DE VIDA DE UM OBJETO

1. **Criação** → instanciado a partir da classe.
2. **Utilização** → acessa atributos e executa métodos.
3. **Destruição** → removido da memória pelo sistema.



VÁRIOS OBJETOS DA MESMA CLASSE

```
Aluno aluno1 = new Aluno();  
Aluno aluno2 = new Aluno();  
  
aluno1.nome = "Carlos";  
aluno2.nome = "Ana";
```

aluno1 e **aluno2** são diferentes objetos, mas da mesma classe Aluno.

OBJETOS NO MUNDO REAL

- Classe: ContaBancaria
- Objetos:
 - conta1 → saldo = 1000
 - conta2 → saldo = 500
- Classe: Aluno
- Objetos:
 - aluno1: nome = João, idade = 22, curso = Computação.
 - aluno2: nome = Maria, idade = 20, curso = Engenharia.
 - aluno3: nome = Ana, idade = 19, curso = Direito.

Ambos seguem a mesma estrutura (Classe), mas têm dados diferentes.

Construtores e Sobrecarga

CONSTRUTORES

- Construtores são **métodos especiais** de uma classe.
- Usados para **inicializar objetos** no momento de sua criação.
- O nome do construtor é **igual ao nome da classe**.
- Não possui tipo de retorno (nem void).
- Pode receber **parâmetros** para inicializar atributos.
- É chamado automaticamente quando o objeto é criado.

```
class Pessoa {  
    String nome;  
    int idade;  
  
    Pessoa(String n, int i) {  
        nome = n;  
        idade = i;  
    }  
}
```

```
Pessoa p1 = new Pessoa("Ana", 25);
```

Criação de um objeto já inicializado.

CONSTRUTORES

Construtor Padrão

- Se nenhum construtor for definido, a linguagem cria um **construtor vazio**.

```
class Pessoa {  
    String nome;  
    int idade;  
}
```

```
Pessoa p = new Pessoa();
```

Construtor Personalizado

- Permite definir **valores iniciais**.

```
class Conta {  
    int numero;  
    double saldo;  
  
    Conta(int n, double s) {  
        numero = n;  
        saldo = s;  
    }  
}
```

EXEMPLO PRÁTICO DE CONSTRUTOR

```
class Conta {  
    int numero;  
    double saldo;  
  
    // Construtor  
    Conta(int n, double s) {  
        numero = n;  
        saldo = s;  
    }  
  
    // Método para exibir informações  
    void exibirDados() {  
        System.out.println("Conta: " + numero +  
                           " | Saldo: " + saldo);  
    }  
}
```

```
public class Main {  
    public static void main(String[] args) {  
        // Criando objetos usando o construtor  
        Conta c1 = new Conta(1234, 500.0);  
        Conta c2 = new Conta(5678, 1500.0);  
  
        // Chamando método para mostrar os dados  
        c1.exibirDados();  
        c2.exibirDados();  
    }  
}
```

Saída Esperada:

```
Conta: 1234 | Saldo: 500.0  
Conta: 5678 | Saldo: 1500.0
```


SOBRECARGA DE CONSTRUTORES

- Uma classe pode ter **vários construtores**.
- Diferenciam-se pela **quantidade** ou **tipo dos parâmetros**.
- Benefícios:
 1. Mais flexibilidade na criação de objetos.
 2. Possibilita inicializações diferentes.
 3. Facilita a reutilização de código.

Uso da Sobrecarga

```
Aluno a1 = new Aluno();  
Aluno a2 = new Aluno("João", 20);
```

- **a1** usa o construtor sem parâmetros.
- **a2** usa o construtor com parâmetros.

Exemplo de Sobrecarga:

```
class Aluno {  
    String nome;  
    int idade;  
  
    Aluno() {  
        nome = "Sem nome";  
        idade = 0;  
    }  
  
    Aluno(String n, int i) {  
        nome = n;  
        idade = i;  
    }  
}
```


SOBRECARGA DE MÉTODOS

- Um método pode ter várias versões.
- Diferença ocorre na lista de parâmetros.

Exemplo de Sobrecarga de Métodos

```
class Calculadora {  
    int soma(int a, int b) {  
        return a + b;  
    }  
  
    double soma(double a, double b) {  
        return a + b;  
    }  
}
```

Vantagens da Sobrecarga

- Mesma operação com diferentes tipos de dados.
- Código mais **organizado e intuitivo**.

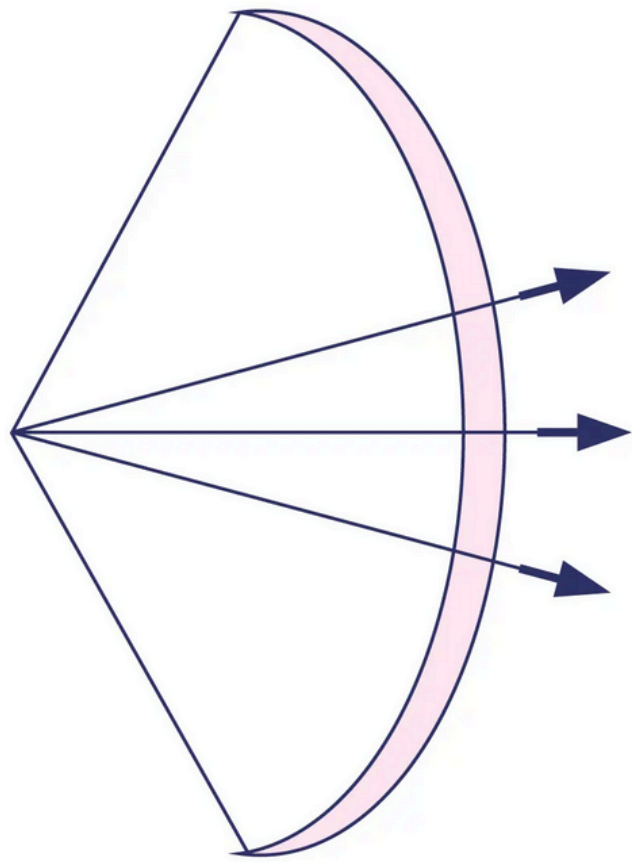
Exemplo Prático

```
Calculadora c = new Calculadora();  
System.out.println(c.soma(3, 4));           // int  
System.out.println(c.soma(2.5, 4.8));       // double
```

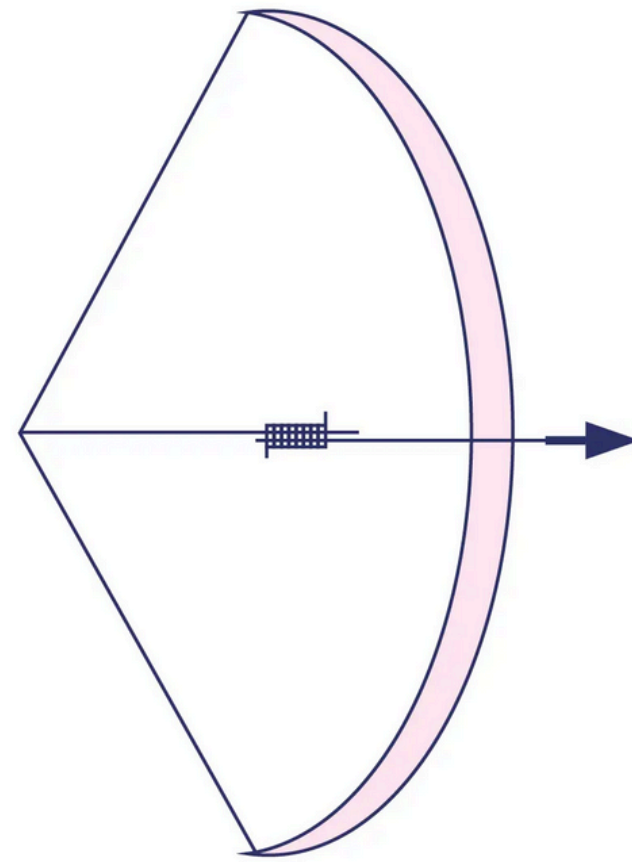
DIFERENÇA ENTRE SOBRECARGA E SOBRESCRITA

- **Sobrecarga (Overloading):** Mesmo nome, parâmetros diferentes, na mesma classe.
- **Sobrescrita (Overriding):** Método redefinido em uma classe filha.

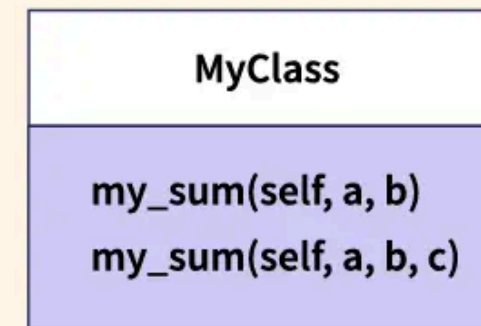
Overloading



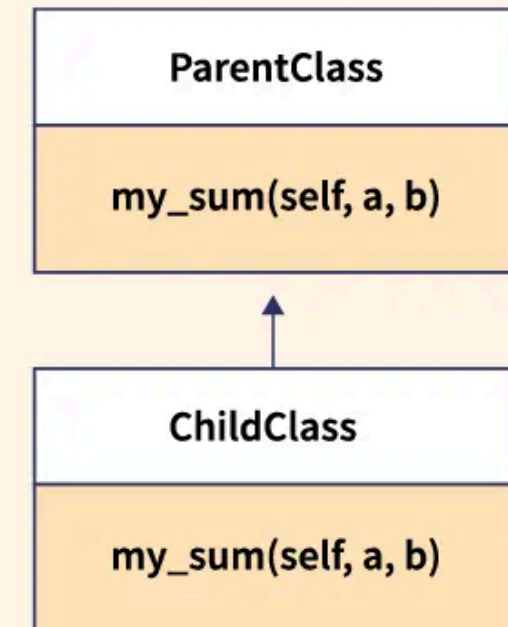
Overriding



Overloading



Overriding



EXEMPLO INTEGRADO

- A classe Produto possui **atributos, construtores e método de exibição**.
1. **Encapsulamento de dados**
(atributos dentro da classe).
 2. **Construtores sobrecarregados**
(permitindo inicialização flexível).
 3. **Métodos** para manipular ou exibir informações.

```
class Produto {  
    String nome;  
    double preco;  
  
    Produto() {  
        nome = "Genérico";  
        preco = 0.0;  
    }  
  
    Produto(String n, double p) {  
        nome = n;  
        preco = p;  
    }  
  
    void mostrar() {  
        System.out.println(nome + " - " + preco);  
    }  
}
```

EXEMPLO INTEGRADO

1) Declaração da classe

```
class Produto {  
    String nome;  
    double preco;  
}
```

- Aqui, estamos criando **uma classe** chamada Produto.
- Ela possui dois atributos:
 1. **nome** → do tipo **String**, que armazena o nome do produto.
 2. **preco** → do tipo **double**, que armazena o preço do produto.

EXEMPLO INTEGRADO

2) Construtor padrão

```
Produto() {  
    nome = "Genérico";  
    preco = 0.0;  
}
```

- Esse é o **construtor padrão**, ou seja, aquele que **não recebe parâmetros**.
- Ele inicializa os atributos com valores padrão:
 1. **nome = "Genérico"**
 2. **preco = 0.0**
- Isso garante que, mesmo sem passar nenhum valor, o objeto **Produto** terá informações válidas.

EXEMPLO INTEGRADO

3) Construtor com parâmetros

```
Produto(String n, double p) {  
    nome = n;  
    preco = p;  
}
```

- Este é um **construtor sobrecarregado**, que **recebe parâmetros** para inicializar os atributos.
- Permite criar um produto **específico**, definindo **nome** e **preco** ao criar o objeto:

```
Produto p = new Produto("Chocolate", 5.50);
```

- Aqui, nome será **"Chocolate"** e **preco** será **5.50**.

EXEMPLO INTEGRADO

4) Método para mostrar informações

```
void mostrar() {  
    System.out.println(nome + " - " + preco);  
}
```

- O método **mostrar()** **imprime os valores** dos atributos do produto no console.
- Exemplo de uso:

```
Produto p1 = new Produto();  
Produto p2 = new Produto("Chocolate", 5.50);
```

p1.mostrar(); // Saída: Genérico - 0.0

p2.mostrar(); // Saída: Chocolate - 5.5

Classes e Objetos

DECLARANDO UMA CLASSE

- Em Java, uma **classe** é um **modelo** (ou "molde") que **define atributos e comportamentos (métodos)** de um **objeto**.

Exemplo simples de uma classe Carro:

- **String marca;** → atributo que guarda a marca do carro.
- **int ano;** → atributo que guarda o ano do carro.
- **void mostrarInfo()** → método que imprime os dados do carro.

```
public class Carro {  
    // Atributos (características)  
    String marca;  
    String modelo;  
    int ano;  
  
    // Método (comportamento)  
    void mostrarInfo() {  
        System.out.println("Marca: " + marca);  
        System.out.println("Modelo: " + modelo);  
        System.out.println("Ano: " + ano);  
    }  
}
```

INSTACIANDO OBJETOS

```
public class Main {  
    public static void main(String[] args) {  
        // Criando um objeto da classe Carro  
        Carro carro1 = new Carro();  
  
        // Acessando e atribuindo valores aos atributos  
        carro1.marca = "Toyota";  
        carro1.modelo = "Corolla";  
        carro1.ano = 2023;  
  
        // Chamando o método do objeto  
        carro1.mostrarInfo();  
  
        // Criando outro objeto  
        Carro carro2 = new Carro();  
        carro2.marca = "Honda";  
        carro2.modelo = "Civic";  
        carro2.ano = 2022;  
  
        carro2.mostrarInfo();  
    }  
}
```

- Um **objeto** é uma **instância de uma classe**. Para **criar um objeto**, usamos a palavra-chave **new** seguida do construtor da classe:

Saída esperada:

```
Marca: Toyota  
Modelo: Corolla  
Ano: 2023  
Marca: Honda  
Modelo: Civic  
Ano: 2022
```

INSTACIANDO OBJETOS: USANDO CONSTRUTORES

- Construtores permitem inicializar objetos de forma mais prática:

```
public class Carro {  
    String marca;  
    String modelo;  
    int ano;  
  
    // Construtor  
    Carro(String marca, String modelo, int ano) {  
        this.marca = marca;  
        this.modelo = modelo;  
        this.ano = ano;  
    }  
  
    void mostrarInfo() {  
        System.out.println("Marca: " + marca + ", Modelo: " + modelo + ", Ano: " + ano);  
    }  
}
```

INSTACIANDO OBJETOS: USANDO CONSTRUTORES

- E no *main*:

```
public class Main {  
    public static void main(String[] args) {  
        Carro carro1 = new Carro("Toyota", "Corolla", 2023);  
        Carro carro2 = new Carro("Honda", "Civic", 2022);  
  
        carro1.mostrarInfo();  
        carro2.mostrarInfo();  
    }  
}
```

- Saída:

```
Marca: Toyota, Modelo: Corolla, Ano: 2023  
Marca: Honda, Modelo: Civic, Ano: 2022
```


ATRIBUTOS X MÉTODOS GET E SET

Atributos

- São as características de um objeto, também chamados de variáveis de instância.
- Geralmente são declarados como **private** para proteger os dados (encapsulamento).
- Exemplo:

```
public class Pessoa {  
    private String nome;    // atributo  
    private int idade;      // atributo  
}
```

ATRIBUTOS X MÉTODOS GET E SET

- **Os Métodos** Servem para acessar e modificar atributos privados de forma segura.
- **Get (getter)**
 - Método que **retorna o valor do atributo**.
 - Convenção de nomenclatura: **getNomeDoAtributo()**

```
public class Pessoa {  
    private String nome;  
    private int idade;  
  
    // Método getter  
    public String getNome() {  
        return nome;  
    }  
}
```

- **Set (setter)**
 - Método que **define ou atualiza o valor do atributo**.
 - Convenção de nomenclatura: **setNomeDoAtributo(valor)**

```
// Método setter  
public void setNome(String nome) {  
    this.nome = nome;  
}  
  
public int getIdade() {  
    return idade;  
}  
  
public void setIdade(int idade) {  
    if(idade >= 0) { // controle simples  
        this.idade = idade;  
    }  
}  
}
```


ATRIBUTOS X MÉTODOS GET E SET

ATRIBUTOS

```
private String nome  
private int idade
```

MÉTODOS GET E SET

```
public String getNome()
```

```
public void  
setNome(String nome)
```

```
public int getIdade()
```

```
public void  
setIdade(int idade)
```

GETTER (GET)

- Serve para **obter/retornar** o valor de um atributo privado.
- Não altera nada, só devolve a informação.
- Exemplo:

```
public int getIdade() {  
    return idade;  
}
```

```
System.out.println(pessoa.getIdade());
```

Mostra a idade da pessoa.

SETTER (SET)

- Serve para **definir ou alterar o valor** de um atributo privado.
- Recebe um parâmetro e atualiza a variável da classe.
- Exemplo:

```
public void setIdade(int idade) {  
    this.idade = idade;  
}
```

```
pessoa.setIdade(25);
```

Altera a idade da pessoa para 25.

GET X SET

- **get** → lê o valor (acesso/leitura).
- **set** → muda o valor (modificação/escrita).

GET

```
public int  
getIdade()
```

SET

```
public void  
setIdade(int  
idade)
```

POR QUE USAR GET E SET?

- **Encapsulamento:** protege os dados do objeto.
- **Validação:** permite adicionar regras (como idade ≥ 0).
- **Flexibilidade:** muda a implementação sem alterar quem usa a classe.

Exemplo de uso:

```
public class Main {  
    public static void main(String[] args) {  
        Pessoa p = new Pessoa();  
  
        p.setNome("Lindenberg"); // set  
        p.setIdade(30);  
  
        System.out.println("Nome: " + p.getNome()); // get  
        System.out.println("Idade: " + p.getIdade());  
    }  
}
```

Saída:

```
Nome: Lindenberg  
Idade: 30
```


PALAVRAS-CHAVE PUBLIC E PRIVATE

Em Programação Orientada a Objetos (POO), **public** e **private** são modificadores de acesso que controlam a visibilidade de atributos, métodos e classes.

Public vs Private In Java

```
private String sayHi() {return "hi";}
public String sayHello() {return "hello";}
```



PALAVRAS-CHAVE PUBLIC E PRIVATE

PRIVATE

- O acesso é **restrito apenas dentro da própria classe**.
- Ninguém fora da classe consegue acessar diretamente.
- Usado para **proteger os dados** (encapsulamento).
- Normalmente aplicado em **atributos**.

Exemplo:

```
public class Pessoa {  
    private String nome;    // só pode ser acessado dentro da classe  
    private int idade;  
  
    // Métodos públicos para acessar atributos privados  
    public String getNome() {  
        return nome;  
    }  
  
    public void setNome(String nome) {  
        this.nome = nome;  
    }  
}
```

Aqui, **nome** e **idade** estão protegidos. Para acessar, é obrigatório usar **get** e **set**.

PALAVRAS-CHAVE PUBLIC E PRIVATE

PUBLIC

- O acesso é **público**, ou seja, pode ser usado em **qualquer lugar do programa**.
- Normalmente aplicado em **métodos** ou **classes principais**.
- Usado quando queremos que algo seja **visível para todo o código**.

Exemplo:

```
public class Main {  
    public static void main(String[] args) {  
        Pessoa p = new Pessoa();  
        p.setNome("Ana");           // acessível  
        System.out.println(p.getNome()); // acessível  
    }  
}
```

Aqui, **setNome()** e **getNome()** são públicos, logo podem ser chamados fora da classe.

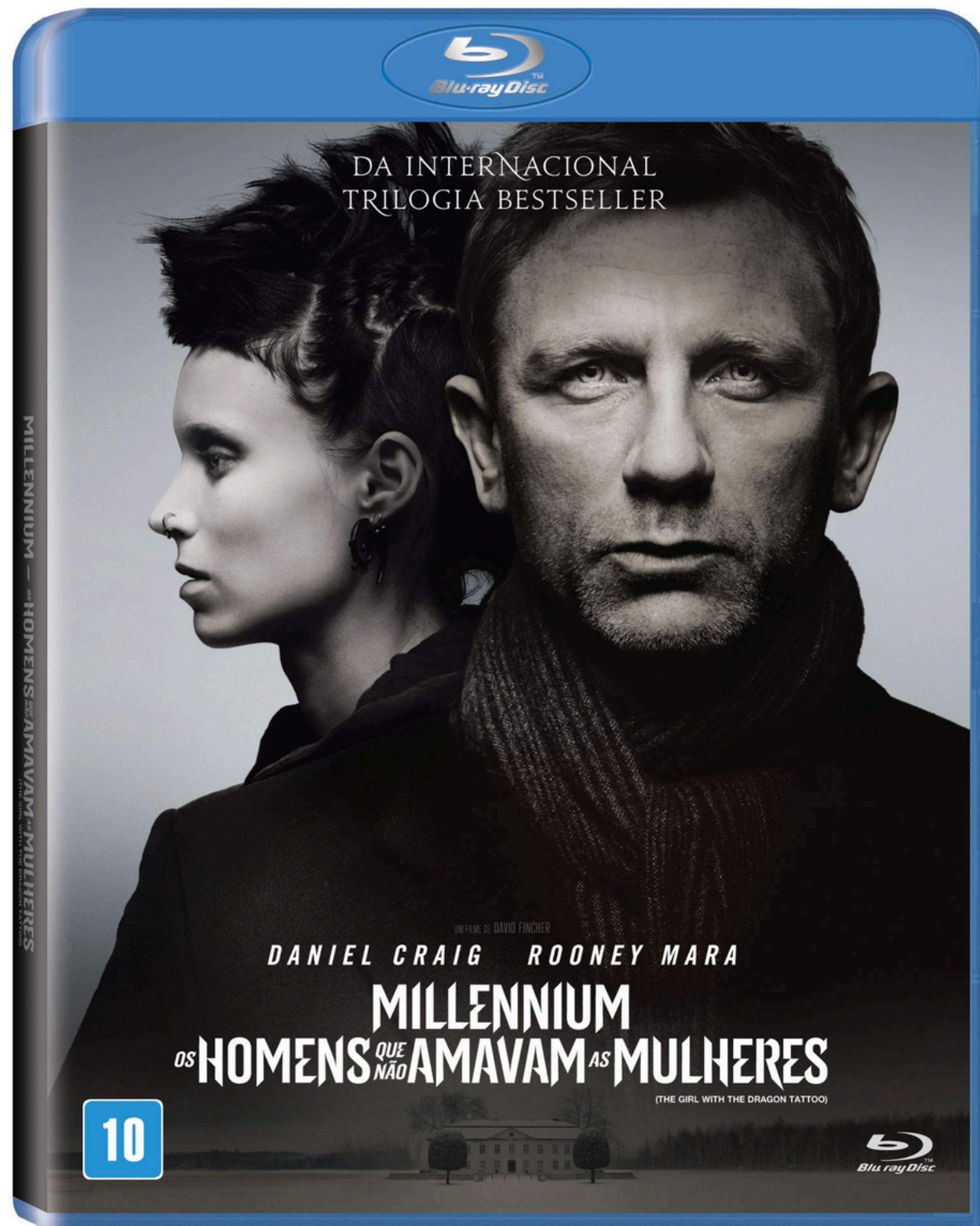
PALAVRAS-CHAVE PUBLIC E PRIVATE

Modificador	Onde pode ser usado?	Quem pode acessar?	Uso comum:
private	Atributos e métodos	Somente a própria classe	Proteção de dados (encapsulamento)
public	Classes, métodos e atributos	Qualquer parte do programa	Métodos de acesso (get/set), classes principais

REFERÊNCIAS

1. BARNES, David J.; KÖLLING, Michael. Programação orientada a objetos com Java: uma introdução prática usando o BlueJ. 4. ed. São Paulo: Pearson Prentice Hall, 2008.
2. SANTOS, Rafael. Introdução à Programação Orientada a Objetos usando Java. 2. ed. Rio de Janeiro: Elsevier, 2013. 336 p.
3. SERSON, Roberto Rubinstein. Programação orientada a objetos com Java 6 – curso universitário. 1. ed. Rio de Janeiro: Brasport, jun. 2009. 492 p.
4. PREISS, Bruno R. Estruturas de dados e algoritmos: padrões de projetos orientados a objetos com Java. Rio de Janeiro: Campus, 2001. 566 p.

Millennium: Os Homens que Não Amavam as Mulheres(2012)



Harriet Vanger (Moa Garpandal) desapareceu há 36 anos, sem deixar pistas, em uma ilha no norte da Suécia. O local é de propriedade exclusiva da família Vanger, que o torna inacessível para a grande maioria das pessoas. A polícia jamais conseguiu descobrir o que aconteceu com a jovem, que tinha 16 anos na época do sumiço. Mesmo após tanto tempo, seu tio Henrik Vanger (Christopher Plummer) ainda está à procura e decide contratar Mikael Bomkvist (Daniel Craig), um jornalista investigativo que trabalha na revista Millennium. Bomkvist, que não está em um bom momento por enfrentar um processo por calúnia e difamação, resolve aceitar a proposta e começa a trabalhar no caso. Para isso, ele vai contar com a ajuda de Lisbeth Salander (Rooney Mara), uma investigadora particular incontrolável e antissocial.

Dúvidas?



Profº Lindenberg Andrade
E-mail: linndemberg1@gmail.com



Additional contacts via QR code